

# From Raw Reviews to Business Intelligence: How I Built a Production-Grade NLP Pipeline for Ethiopian Banks

---

*A technical walkthrough of transforming a Week 2 data project into a finance-sector portfolio piece*

---

## The Problem I Set Out to Solve

Three of Ethiopia's largest banks — CBE, BOA, and Dashen — have millions of customers using their mobile apps. Every day, those customers leave reviews on Google Play. Some praise the app. Many complain. And the banks have no systematic way to know what those complaints are actually about.

I built a system that reads all of those reviews, classifies the sentiment, identifies the complaint themes, and presents the findings in an interactive dashboard — automatically.

But this post is not just about what I built. It is about *how* I built it, and why the engineering choices matter as much as the model accuracy when you are trying to impress a finance sector employer.

---

## Where I Started: A Working but Fragile Project

In Week 2, I built the original version of this project. It worked. I had a scraper that collected 1,502 reviews, a DistilBERT sentiment model that classified each one, a theme extraction module, and a PostgreSQL database. The notebooks had charts.

But when I looked at it honestly for Week 12, I saw the problems:

**No tests.** If I changed anything in the preprocessing code, I had no way to know if I had broken something.

**Hardcoded credentials.** The database password was written directly in the source code and committed to a public GitHub repository. This is a serious security failure that would be caught immediately in any professional code review.

**No type hints.** Functions had no documentation of what they accepted or returned. A new team member — or me, six months later — would have to read through the implementation to understand the interface.

**No CI/CD.** There was a GitHub Actions workflow file, but it only installed dependencies. It never actually ran the tests.

**No interactive dashboard.** The results lived in Jupyter notebooks. A non-technical stakeholder — a bank product manager, a risk analyst — could not explore the data themselves.

The code did the job, but it was not production-ready. This week I fixed that.

---

## The Refactoring: Making the Code Professional

The first thing I did was restructure the entire project. Everything that does work went into `src/`. Everything that tests that work went into `tests/`. The dashboard got its own directory. Data files stay in `data/` and are never committed to GitHub.

### Configuration with Dataclasses

I created `src/config.py` to centralise all settings:

```
@dataclass
class SentimentConfig:
    model_name: str = "distilbert-base-uncased-finetuned-sst-2-english"
    threshold: float = 0.7
    batch_size: int = 32
```

Before this, thresholds were magic numbers scattered across different files. Now if I want to change the confidence threshold, I change it in one place and it propagates everywhere.

### Fixing the Security Problem

The original database connection looked like this:

```
engine = create_engine("postgresql://postgres:selfmade@localhost:5432/bank_1")
```

A real password, in plain text, in a public repo. I replaced it with:

```
DatabaseConfig(
    host=os.getenv("DB_HOST", "localhost"),
    password=os.getenv("DB_PASSWORD", ""),
)
```

Now credentials live in a `.env` file that is gitignored. The code is safe to share publicly.

## Adding Type Hints Everywhere

Every function now declares what it takes and what it returns:

```
def assign_theme(text: str) -> str:
def run_sentiment(df: pd.DataFrame, model: Pipeline, batch_size: int = 32) -
def filter_banks(df: pd.DataFrame, banks: Optional[List[str]] = None) -> pd.
```

This is not just style. It enables IDE autocompletion, catches type errors before runtime, and serves as living documentation.

---

## The Tests: Where I Found a Real Bug

I wrote 61 tests across 5 test files. Not because I was told to write tests, but because writing them immediately revealed a problem I did not know existed.

In `theme_extraction.py`, the `clean_text()` function had this line:

```
df["review"] = df["review"].astype(str).str.lower()
```

When `review` is `None`, `astype(str)` converts it to the string `"nan"` rather than an empty string. The test I wrote expected an empty string:

```
def test_handles_nan(self):
    df = self._make_df([None])
    result = clean_text(df)
    assert isinstance(result["review"].iloc[0], str)
```

The test failed. I fixed the function:

```
df["review"] = df["review"].fillna("").astype(str).str.lower()
```

Now it handles null reviews correctly. Without the test, this bug would have silently corrupted downstream sentiment analysis on any reviews where the scraper returned null content. That is exactly why finance employers value tested code — not because tests are a checkbox, but because they catch real problems.

---

## The Dashboard: Turning Analysis into a Tool

The biggest qualitative change was building the Streamlit dashboard. This shifted the project from "a data scientist ran some analysis" to "here is a tool you can use."

The dashboard has five tabs:

**Overview** shows four KPI cards the moment it loads: total reviews, percentage positive, average rating, and the top complaint category. A bank manager can see the summary in ten seconds.

**Sentiment** shows a pie chart of positive vs. negative overall, and a grouped bar chart comparing all three banks side by side. CBE's significantly higher negative rate is immediately visible.

**Ratings** shows average rating per bank and the full distribution of 1–5 star ratings. You can see that CBE has a heavy concentration of 1-star reviews.

**Themes** shows which complaint categories appear most frequently. Account Access Issues dominate for CBE.

Transaction Performance issues are the main pain point for BOA.

**Reviews** is a searchable table where anyone can filter by bank and sentiment and read the actual review text.

The sidebar filters (bank selector, sentiment toggle, rating range slider) update all charts in real time.

Critically, the dashboard ships with a demo mode. It generates synthetic data automatically if you have not run the pipeline. This means I can demonstrate it to anyone without first setting up PostgreSQL and running the full NLP pipeline on their machine.

---

## The Results: What the Data Actually Says

After running the full pipeline on 1,502 reviews, here is what emerged:

**Dashen Bank** has the highest customer satisfaction — approximately 68% positive reviews and an average rating of 3.7 out of 5. Its primary complaints are about UI and navigation, which are relatively straightforward to address.

**Bank of Abyssinia (BOA)** sits in the middle with 54% positive reviews and an average rating of 3.2. Transaction Performance — slow transfers and payment delays — is its biggest issue. This suggests backend infrastructure constraints rather than app-level problems.

**Commercial Bank of Ethiopia (CBE)** has the most room for improvement. Only 41% of its reviews are positive, and its average rating is around 2.8. The dominant complaint category is Account Access Issues — login failures,

password resets, OTP problems. Given CBE's enormous customer base, this is a high-priority operational problem.

**The business recommendation is clear:** CBE should direct engineering resources specifically at the authentication and login flow. A 10-percentage-point reduction in login-related complaints would likely push the average rating above 3.0, which is the threshold most customers use to decide whether to recommend an app.

---

## Model Explainability: The SHAP Layer

For finance sector work, "the model said so" is not sufficient. Decisions need to be auditable.

I added a SHAP explainability module ( `src/explainability.py` ) that provides two levels of explanation:

**Global importance** shows which words most characterise negative reviews across the entire dataset. Terms like "crash", "delay", "failed", "login", and "error" consistently appear at the top. This gives a macro-level view of what is driving dissatisfaction.

**Local explanation** uses SHAP token-level attribution to explain individual predictions. For a review like *"The app keeps crashing and my transfer was delayed"*, SHAP highlights "crashing" and "delayed" as the primary negative signals. This lets a product manager look at any flagged review and understand exactly why the model classified it as negative.

This kind of transparency is standard practice in financial institutions, where model decisions may need to be explained to regulators or customers.

---

## The CI/CD Pipeline: Proving Quality Automatically

The GitHub Actions workflow now runs on every push:

1. Install dependencies (pinned versions, no version drift)
2. Run flake8 linting
3. Run all 61 pytest tests
4. Check that test coverage is at least 60%

If any step fails, the push is flagged. The CI badge in the README shows the current build status.

This matters for a finance portfolio for one reason: it proves that the code works, automatically, on a clean machine that has never seen this project before. It is the equivalent of showing your work in an exam — not just the answer, but the evidence.

---

## What I Learned

**Testing catches real bugs.** The NaN bug in `clean_text` was invisible until a test exposed it. Bugs like that — silent data corruption — are the most dangerous kind.

**Configuration discipline pays off.** Having all settings in `src/config.py` meant I could change the sentiment model, update file paths, or swap database credentials without touching any pipeline code. This is how production systems work.

**A tool beats a report.** The dashboard changed what I could say about this project. Instead of "I ran analysis and



here are the charts", I can say "here is a live tool — filter by CBE, select negative reviews, look at the themes yourself."

**Security is not optional.** Hardcoded credentials in a public repo is not a minor issue. It is the kind of thing that fails a code review at any serious organisation. Fixing it was non-negotiable.

---

## What Is Next

The project is solid, but there is more to do:

- **Fine-tune on domain data.** A model trained specifically on Ethiopian banking reviews would outperform the general-purpose DistilBERT.
  - **Amharic support.** Many customers write in Amharic. XLM-RoBERTa handles multilingual sentiment and would capture a much larger share of feedback.
  - **Automated weekly runs.** Schedule the scraper and pipeline via GitHub Actions cron to keep the dashboard current.
  - **Power BI export.** Many Ethiopian financial institutions use Power BI for executive reporting. Adding a connector would make this tool immediately deployable.
- 

## The Code

Everything is open source and documented:

**GitHub:** <https://github.com/Dagi0808/fintech-review-analytics>

```
git clone https://github.com/Dagi0808/fintech-review-analytics.git
cd fintech-review-analytics
pip install -r requirements.txt
streamlit run dashboard/app.py
```

The dashboard works immediately with demo data. Run `python -m src.main` after scraping to load real reviews.

---

*Built as part of the 10 Academy Week 12 Capstone. The goal was to transform a working project into a production-ready portfolio piece — and in the process, learn what the difference actually looks like in code.*